

Patchright Enhanced

配置与生命周期小白指南

12 个问题与 2026-10-05 历史实验：默认值、数值解析和异常清理。公开整理未重跑实验

20/20

当前行为断言匹配
包含已暴露的缺陷，不等于项目健康评级

本轮结论

- 1. 空环境的并发默认值为 2；README 与示例环境写的是 1
- 2. 非法并发字符串没有被完整校验，可能变成 NaN 或截断整数
- 3. 关闭拒绝会跳过清理，必须单独设计失败分支验收

实验边界

真实执行 3 个上游模块；未安装或导入 Patchright，未启动浏览器，未访问目标网页，未执行完整构建。源码读取与包内复现脚本的 Git 下载属于来源获取，不属于浏览器实验。

疑问即目录

- | | |
|--------------------------------|------------------------------|
| 01 Patchright Enhanced 到底包住了什么 | 07 为什么关闭要 await，清理却是同步调用 |
| 02 读到网页文本就算浏览器跑通了吗 | 08 关闭抛错后，目录仍一定会清理吗 |
| 03 从入口到关闭会经过哪些对象 | 09 为什么还要测试重复关闭和失败后再关闭 |
| 04 README 的默认值就是实际默认值吗 | 10 不安装 Patchright，测试的还是上游代码吗 |
| 05 MAX_PARALLEL 写成 oops 会被拒绝吗 | 11 看到源码矛盾，为什么不能直接写构建失败 |
| 06 Context 和 Page 为什么不是一回事 | 12 怎样判断我已经学会，而不是只看了绿灯 |

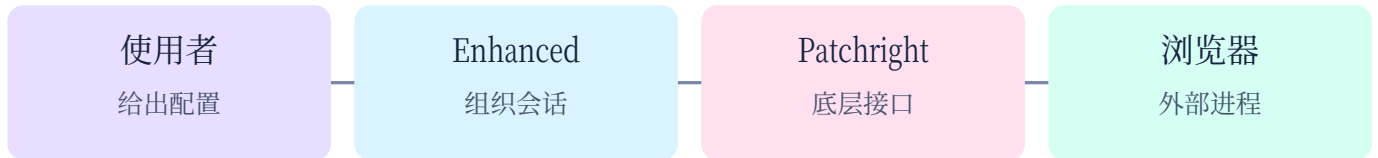
阅读 02-07 页 / 能力与用途 08 页 / 六步实验 09-11 页 / 来源与验收 12 页

固定提交 e38ab7ab9448db6f093f72ee097c18ca9905e84c

01

Patchright Enhanced 到底包住了什么

它把配置和会话组织起来，真正的浏览器仍由底层库创建



wrapper 就是包装层：在已有库外，补上常用的配置、调用次序和资源管理。这个 TypeScript 项目依赖 Patchright；仓库名是 patchright-enhanced，package.json 的包名却是 ghostprobe，检索时要对准仓库与提交。

像实验室里接好的仪器架：机架让操作顺手，却不等于换了测量仪器。这里的机架是 SessionManager、SessionRunner、BrowserManager；底层调用位于 context-factory.ts。

与 Agent-Reach 公开笔记只作一层衔接：工具路由负责选工具，本项目负责自己的配置与会话。两者不是同一层。

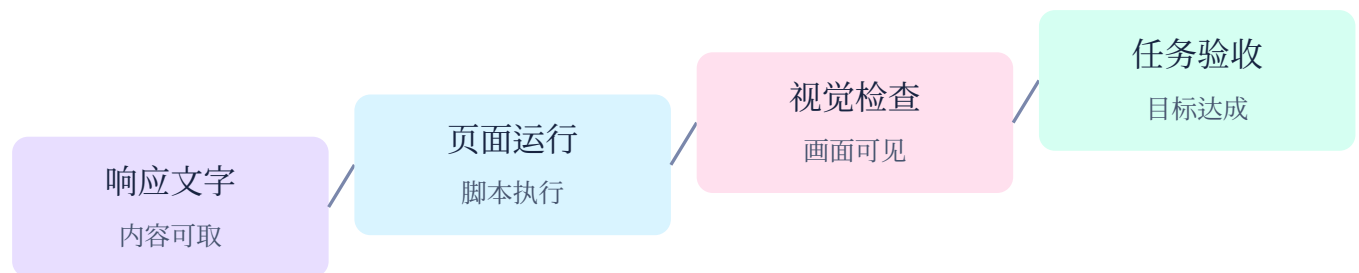
[证据 源码阅读；没有运行底层浏览器](#)

来源 package.json L1-L18 / src/browser/context-factory.ts L13-L40

02

读到网页文本就算浏览器跑通了吗

检索、渲染和完成任务，各需要自己的证据



HTTP 检索是取得响应文字；浏览器渲染是执行页面环境并画出画面；任务成功还要求结果满足需求。取得 HTML 不证明脚本执行，也不证明按钮能点。

例如课程页面：能读出“提交”两个字，只证明文本存在；看到按钮，仍不能证明已提交。本轮只运行本地包装层单元测试，连真实浏览器启动这一关都没有验证。

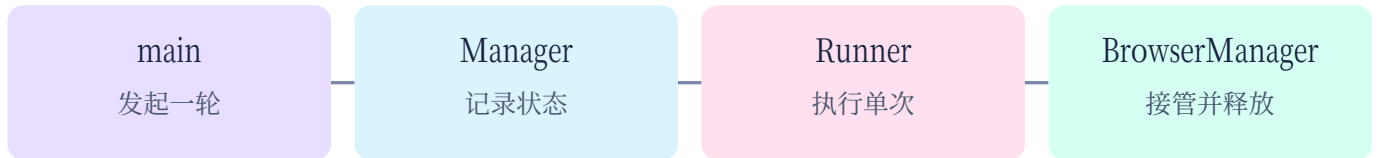
沿用“证据优先质检”：每个结论注明证明到了哪一层。图中的台阶是不同验收项，不是自动通关。

[证据 本轮证据范围说明](#)

03

从入口到关闭会经过哪些对象

会话调度、单次运行和资源关闭各有主人



main.ts 读配置后反复发起一轮会话；SessionManager 为单次运行建立状态；SessionRunner 调用创建函数、打开起始页，并等待关闭事件；BrowserManager 保存 context 与临时目录的引用。

生命周期指资源从创建、使用到释放的过程。这里需要特别看“何时接管”：只有 createBrowserContext 返回后，setContext 才登记资源；创建途中出错是否善后，不能靠后面的 finally 想当然。

已有“工具调用生命周期”谈的是 MCP 调用循环；这里谈资源所有权，仅关联，不同义合并。

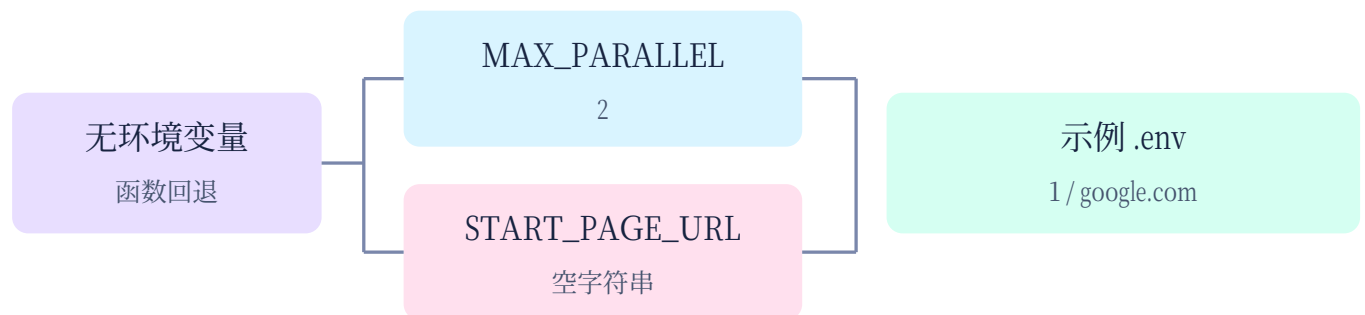
证据 源码调用图；SessionRunner 与工厂未执行

来源 main.ts L8-L36 / src/session/session-runner.ts L25-L55

04

README 的默认值就是实际默认值吗

没有环境变量时，MAX_PARALLEL 是 2，起始 URL 是空字符串



默认值是调用者没有提供值时程序选用的值。README 与 .env.example 写的是 MAX_PARALLEL=1、START_PAGE_URL=https://google.com/；源码直接调用时分别回退到 2 和空字符串。

两者能同时存在：把 .env.example 复制成 .env，且通过 main.ts 加载 dotenv，是一条输入路径；直接调用配置函数且环境为空，是另一条路径。要记录前提，不能把示例配置当成函数默认值。

实验会分别显示空环境与显式输入。这里不是文档改错实验，也不会启动页面。

证据 A01/A09 实测并发默认值；起始 URL 仅源码确认

来源 readme.md L27-L36 / src/config/index.ts L7-L21 / src/start/default.ts L4-L6

05

MAX_PARALLEL 写成 oops 会被拒绝吗

parseInt 是转换函数，不是配置校验器

正整数	尾部字母	非数字	负整数
"3" 3	"2junk" 2	"oops" NaN	"-1" -1

parseInt 把字符串开头能识别的整数读出来；NaN 表示“不是一个有效数字”。源码没有追加正整数、范围或完整字符串检查。实验矩阵会展示 3、0、负数、带字母后缀和完全非数字输入的区别。

例如“2junk”能被读成 2，却不意味着“2junk”是合法并发配置。调用端的循环可能因此走不同路径；本轮验证的是解析结果，不是实际并发性能或完整主循环。

改一个输入立即看到输出变化，是本实验的中间成果。若用于真实工程，应另加有限、正整数和上限校验。

证据 A02/A04-A09 实测输入矩阵

来源 src/config/index.ts L19-L20 / src/session/session-manager.ts L59-L67

06

Context 和 Page 为什么不是一回事

Context 是会话环境，Page 是其中的一张页面



BrowserContext 表示浏览器会话环境；Page 表示页面对象。此项目调用 launchPersistentContext(userDataDir, options) 创建带用户数据目录的会话，再调用 context.newPage()。持久目录可承载会话状态，不能把“新 Page”直接理解成“全新身份”。

这里的工厂返回 context、page、tempDir 三件东西。就像打开同一工作间里的另一个窗口：窗口多了，工作间仍是同一个。隔离效果、目录内容与实际浏览器行为，本轮都没有端到端测量。

不要与“健康探测三态”混淆：missing/broken/timeout 是诊断分类，不是 Browser/Context/Page 三层对象。

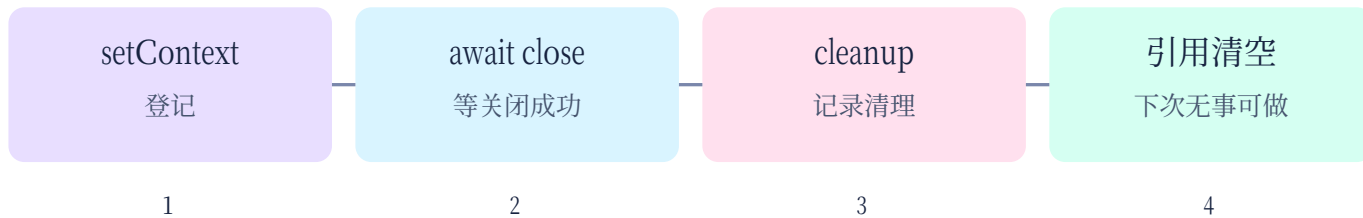
证据 源码阅读；浏览器隔离行为未验证

来源 src/browser/context-factory.ts L6-L10 / src/browser/context-factory.ts L17-L40

07

为什么关闭要 await，清理却是同步调用

先等关闭完成，再清理目录；顺序本身就是契约



异步操作用 Promise 表示尚未完成的结果；await 等待它结束或抛出错误。BrowserManager.close() 等待 context.close()，成功后才把引用设为 null；cleanup() 调用目录清理函数后清空路径引用。

closeAndCleanup() 顺序调用两者。实验的假 context 只记录 close，假的清理函数只记录 cleanup；它们不打开浏览器、不删除目录，却能证明真实管理器调用了谁、先后次序怎样。

“接缝与桩实现”在这里落地：换掉危险的外部效果，保留真正要检查的管理器逻辑。

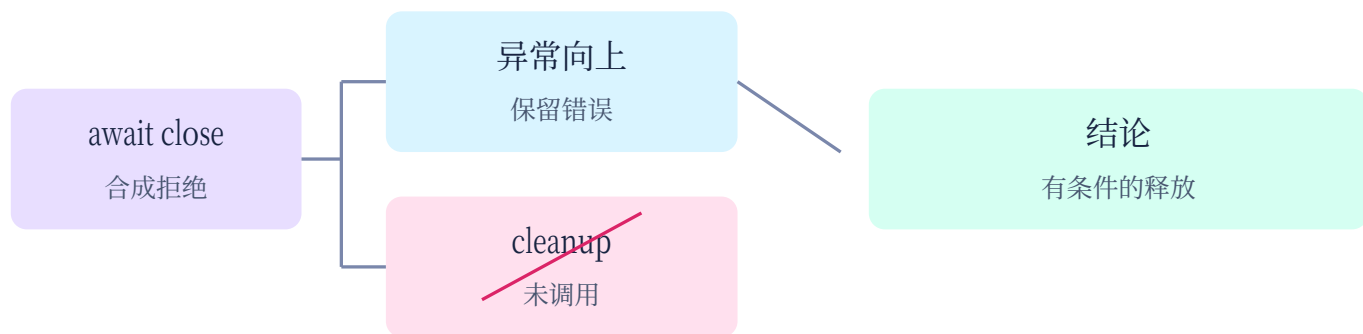
证据 L03/L04 实测关闭与清理次序

来源 src/browser/browser-manager.ts L13-L31

08

关闭抛错后，目录仍一定会清理吗

当前顺序里，close 拒绝会跳过后面的 cleanup



拒绝是 Promise 的失败结果。closeAndCleanup() 没有自己的 try/finally；若 await close() 抛出错误，执行流直接离开函数，cleanup() 不会被调用。故障注入就是让替身按预定方式失败，再观察真实代码。

实验用合成错误证明这个分支，绝不声称实测了浏览器崩溃。修复思路可考虑 finally 中处理清理，并分别保留关闭与清理错误；但也要定义“关闭未完成能否删目录”，本教材不修改上游。

失败不是一句“没跑通”：要写清发生层、异常如何传播、哪些后续动作没有发生。

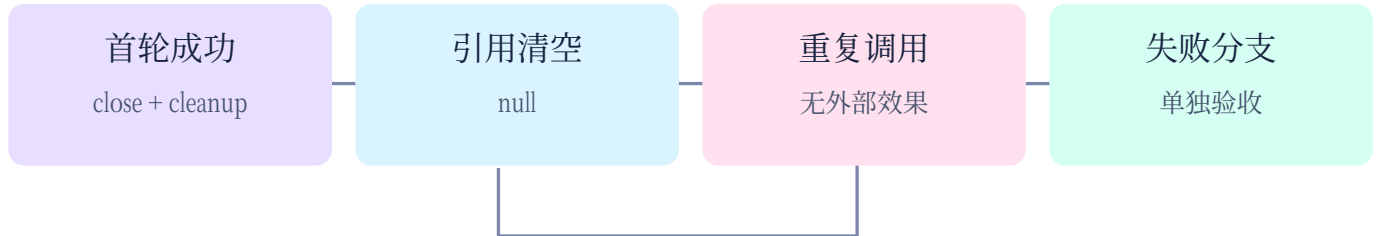
证据 L06 实测关闭拒绝与清理跳过

来源 src/browser/browser-manager.ts L13-L31 / src/session/session-runner.ts L53-L55

09

为什么还要测试重复关闭和失败后再关闭

一次成功看不到资源管理的边角行为



幂等性指同一动作重复执行不会产生额外效果。成功关闭并清理后再次调用，引用已经清空，理应没有第二次外部调用；关闭先失败、下一次成功时，路径是否还在，则决定后续能否清理。

还有所有权问题：setContext 只替换两个字段，本身不关闭旧资源。若重复登记新 context，调用者必须知道旧对象由谁负责。这里只验证能被桩观察到的调用行为，不把局部结果升级为“所有异常都安全”。

L05 确认成功后重复调用无额外效果；L07 确认关闭失败后的一次合成重试可继续清理。清理函数自身抛错尚未测试。

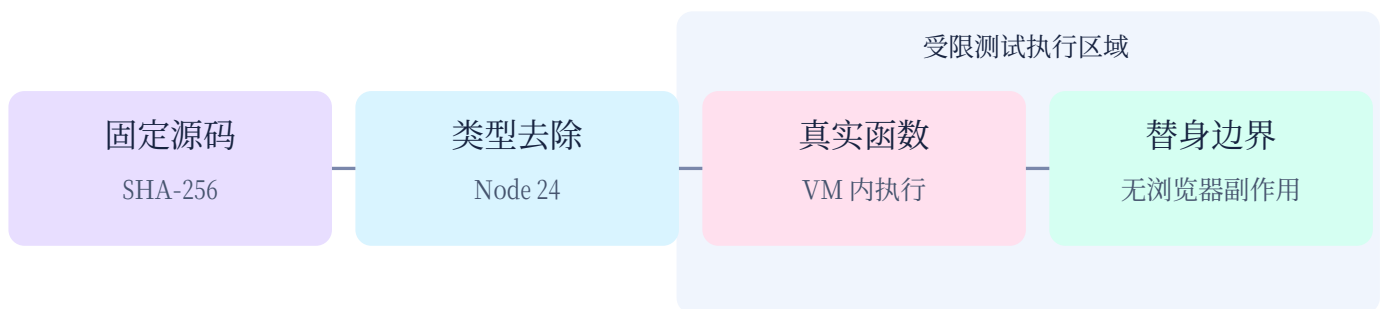
证据 L05/L07 实测；重复登记为源码观察

来源 src/browser/browser-manager.ts L8-L31

10

不安装 Patchright，测试的还是上游代码吗

运行固定的真实函数，替换边界依赖，并记录改写范围



本地测试夹具读取指定提交的源码，由现有 Node 24 的 stripTypeScriptTypes 去除类型语法，再用受限 VM 执行选定函数。VM 是提供独立执行上下文的机制；这里的防线是白名单代码与显式依赖替换，不把 VM 当成处理任意恶意代码的安全沙箱。

文件系统、代理配置与清理效果由惰性替身承接，路径只作纯运算；动态导入和真实浏览器路径不参与。源码散列固定输入，测试日志固定观察结果。这样可检查局部逻辑，却不能证明包安装成功、类型检查通过或网页可用。

这是已有“接缝与桩实现”的应用，不另造同义概念。

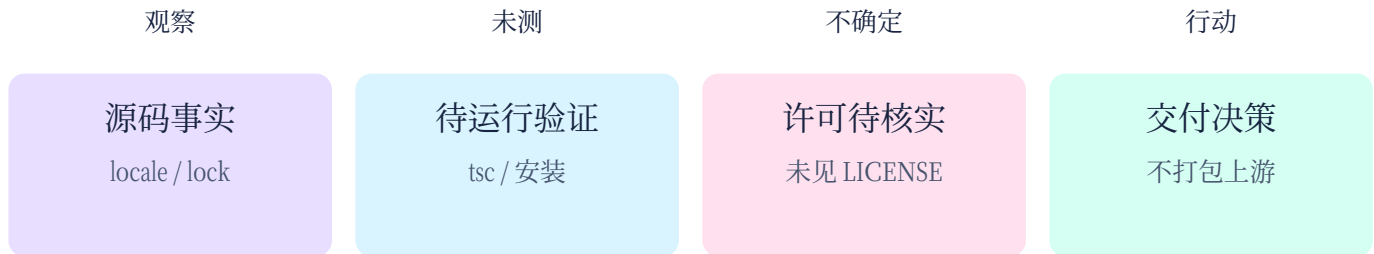
证据 自建 harness 执行记录；未做 tsc 编译

来源 src/config/index.ts L1-L21 / src/browser/browser-manager.ts L1-L36

11

看到源码矛盾，为什么不能直接写构建失败

静态疑点、编译结果和运行结果必须分栏



BrowserConfig 把 locale 声明为必填，但 getBrowserConfig 的 locale 行被注释；package.json 与 pnpm-lock.yaml 的依赖条目也不一致。这些都是该提交可定位的源码事实。

本轮没有可用 tsc，也没有安装上游依赖，因此没有编译器报错日志，不能写“构建已失败”。同样，仓库跟踪文件中未发现 LICENSE，不代表可以自由再分发或商用；应先核实许可。练习包只发自建脚本与源码指纹。

沿用“开源验货三查”，但不把旧笔记中的许可结论直接迁移到当前项目。

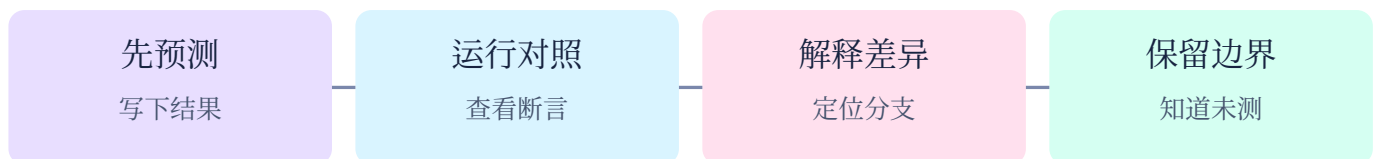
证据 源码与文件树检查；未编译，许可未确认

来源 src/types/config.types.ts L1-L6 / src/config/browser.config.ts L3-L10 / package.json L11-L19 / pnpm-lock.yaml L7-L32

12

怎样判断我已经学会，而不是只看了绿灯

能预测新输入、解释失败，并准确说出未测范围



先预测空输入与“2junk”的结果，再跑数值矩阵；先画出关闭失败的执行流，再对照生命周期日志。若能解释“为什么有的 assert 通过是在确认一个缺陷”，就比只看 PASS 多走了一步。

完整证据链包括仓库地址、提交、文件散列、环境版本、执行命令、原始结果、限制与独立审核。验收项为：练习可复现、PDF 可读、结论可回溯；真实浏览器、站点通行、性能和任务完成仍无证据。

后续真实浏览器测试须单列验收记录并限于自有或明确授权系统；不能用这次历史单元测试记录替代。

证据 课程验收清单

这个项目给我什么

先分清源码提供了什么，再看本轮到底验证了什么

- 1 集中读取配置**
getConfig、getBrowserConfig、getStartPageUrl 形成可检查入口。前两个函数实测；起始 URL 仅源码；缺少完整输入校验
- 2 组织多会话调用**
SessionManager 创建状态并聚合 Promise。仅源码；没有测浏览器并发与性能
- 3 封装 Context 与 Page 创建**
工厂返回 context、page、tempDir。仅源码；未调用真实工厂
- 4 管理关闭与清理**
BrowserManager 暴露 close、cleanup 与组合方法。本轮真实管理器测试；关闭失败清理有缺口
- 5 集中记录会话结果**
SessionRunner/Manager 返回成功或失败信息。仅源码；finally 的异常传播仍需单独覆盖

四个可以马上迁移的用途

合法测试项目验收

先检查自有应用自动化脚本的配置和资源管理，避免一上来就启动大量进程

把隐式默认值写进测试

用空值、显式值和错误值检查开发机与持续集成环境的差异

给资源封装补回归用例

用关闭失败、再次关闭、重复调用识别回收契约的漏洞

做依赖接入前的技术评估

保留版本指纹、许可待核实项和未测层级，再决定是否开展单独授权的集成验证

前提：仅用于自有或明确授权的系统。本教材没有验证隐匿、绕过访问限制或网站通行效果。

动手实验 1 至 2 步

先解压练习 ZIP，进入其中 patchright-enhanced-exercise 目录。以下命令适用于 Bash。原学习六步重放与当前副本核验分别见独立审核日志。

1 检查环境

```
node --version  
git --version
```

目的 确认现有工具，不安装上游依赖

实现与观察 本轮为 Node v24.19.0 与 Git 2.52.0

验证成功 版本命令退出码 0；若没有 Node 24，先停止，不用别的方式冒充本轮实验

2 获取固定来源并执行

```
bash reproduce.sh work
```

目的 正常公共 Git 读取后运行 3 个真实模块，外部效果用替身

实现与观察 会生成 work/results.json 和 work/reproduce.log

验证成功 20/20 assertions matched；3 个 SHA-256 一致。work 必须是新目录，已存在时脚本会拒绝覆盖

提示：耗时仅为这组单元断言运行时间，不是浏览器性能。ExperimentalWarning 保留在原始日志；去类型语法不等于 TypeScript 类型检查。

动手实验 3 至 4 步

沿用前面的工作目录和 work/results.json。查看器只读结果，不会再启动上游模块。

3

查看默认值和显式配置

```
node inspect.mjs work/results.json defaults
```

目的 比较空环境、显式环境、空字符串与不存在的开关

实现与观察 A01 的 maxParallel 为 2；B02 显式时区为 Etc/UTC；B04 的 HEADLESS=true 未改变 false

验证成功 A02 只记录递归建目录请求，realFilesystemChanged=false；这不是实际目录创建测试

4

查看错误输入矩阵

```
node inspect.mjs work/results.json counts
```

目的 观察转换成功与输入有效之间的差距

实现与观察 0→0；-1→-1；oops→NaN；2junk→2；2.9→2；空字符串→2

验证成功 A04-A08 的 rejected=false；PASS 表示准确捕获当前缺少校验的行为

提示：耗时仅为这组单元断言运行时间，不是浏览器性能。ExperimentalWarning 保留在原始日志；去类型语法不等于 TypeScript 类型检查。

动手实验 5 至 6 步

沿用前面的工作目录和 work/results.json。查看器只读结果，不会再启动上游模块。

5

检查关闭的三种路径

```
node inspect.mjs work/results.json lifecycle
```

目的 比较正常顺序、成功后重复调用和拒绝后再关闭

实现与观察 L04/L05：close 后 cleanup；L06：cleanupCalls=0；L07：第二次关闭成功后才清理

验证成功 错误 UNIT_CLOSE_FAILURE 是合成故障；未测真实浏览器崩溃，也未测 cleanup 自身抛错

6

核对证据和边界

```
node inspect.mjs work/results.json scope
```

目的 确保正确版本、全部断言及未测声明一起保留

实现与观察 VERIFIED 20/20; 3 source hashes; no browser/navigation/build

验证成功 摘要断言全部通过；结果标志 browserLaunched/navigationPerformed/fullProjectBuildPerformed 均为 false

提示：耗时仅为这组单元断言运行时间，不是浏览器性能。ExperimentalWarning 保留在原始日志；去类型语法不等于 TypeScript 类型检查。

把结论带回证据

源码锚点

官方仓库 [whaleyxb/patchright-enhanced](#)；固定提交 `e38ab7ab9448db6f093f72ee097c18ca9905e84c`。每题提供固定版本源码链接；三份执行文件另有 SHA-256 指纹。

初次实际运行

2026-10-05 08:13:42 UTC, Node v24.19.0, 20 个当前行为断言全部匹配：4 个浏览器配置、9 个应用配置、7 个生命周期用例。具体输入、输出与警告见实验日志。

可复现范围

练习 ZIP 仅有自建 harness、结果查看器、复现脚本、来源指纹和说明。正常 Git 读取后取得固定提交；未将上游源码再分发到 ZIP。独立审核另有第六份日志。

明确未测

真实依赖安装、完整 tsc 构建、浏览器进程、页面导航、视觉页面、代理连接、性能、目标站点和最终业务任务均未验证。START_PAGE_URL 空回退、locale 接口矛盾和 lock 不一致属于源码观察。

知识查重范围

原学习固定公开索引有 394 条元数据；实际全文核对 20 篇相关公开笔记，其余 374 篇仅元数据检索。此范围不代表全库全文阅读，也不代表读者已经掌握。

产物检查方式

当前 PDF 由 ReportLab 直接生成 A4，使用 Poppler 全页渲染。HTML 保留 12 张原创内联 SVG 与打印样式；HTML 浏览器视觉与 HTML 转 PDF 原路线受阻，本次未重试。

下一步判断

固定提交未见跟踪的许可证文件，不能推导为自由再分发或商用授权。后续集成测试应先核实许可与依赖，仅限自有或明确授权的系统；本教材未验证浏览器集成。

打开固定版本仓库

独立审核结果以随包审核日志为准；任何未执行的验收项都应继续标为未验证。